



多态

北京理工大学计算机学院
金旭亮

多态（ polymorphism ）的概念



- “多态性”一词最早用于生物学，指同一种族的生物体虽然具有相同的本质特性，但在不同的具体环境下又有可能呈现出不同的特性。
- 举个例子：北方的玉米普遍长得比南方的长和粗。
- 在面向对象理论中，多态性的定义是：同一操作，作用于不同的类的实例，不同的类将进行不同的解释，最后产生不同的结果。
- 上述文字过于专业，不好理解。换个简单点的通俗一点的说法：

利用多态特性，看上去完全一样的一条语句，在不同的运行环境中，却可以产生不同的运行结果。

掌握多态概念的关键点

多态代码的最本质特征就是——父类（或接口）变量可以引用子类（或实现了接口的类）对象。换句话说：**子类对象可以被当成基类对象使用。**

其典型代码如下所示：

```
//父类变量引用子类对象  
Parent p = new Child();  
  
//MyClass类实现了IMyInterface接口  
IMyInterface obj = new MyClass();
```



总是可以让更一般的对象引用更具体化的对象。



特别地，Java类库的最顶层类是Object。因此每个对象都可以赋值给Object变量。

Java中子类与基类变量间的赋值



子类对象可以直接赋给基类变量。



基类对象要赋给子类对象变量，必须执行类型强制转换，其语法是：

子类对象变量=(子类名)基类对象变量；



也不能乱转换。如果类型转换失败Java会抛出以下这种异常：

ClassCastException

“类型转换”知识点考核



现在有三个类：

```
class Mammal{}  
  
class Dog extends Mammal {}  
  
class Cat extends Mammal {}
```

针对每个类定义三个变量并进行初始化：

```
Mammal m = null ;  
  
Dog d = new Dog();  
  
Cat c = new Cat();
```

下列语句哪一个将引起编译错误？
为什么？哪一个会引起运行时错误？
为什么？

```
m = d;  
d = m;  
d = (Dog)m;  
d = c;  
c = (Cat)m;
```

先进行自我判断，得出结论后，运行 TestCast.java 实例代码，看看你的判断是否正确。

怎样判断对象是否可以转换？

- 可以使用 `instanceof` 运算符判断一个对象是否可以转换为指定的类型：

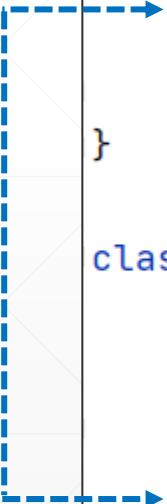
```
Object obj = "Hello";  
if ( obj instanceof String )  
    System.out.println("obj对象可以被转换为字符串");
```

参看实例: TestInstanceof.java

在实践中理解把握复杂的知识-1

- 请看以下“变态”的类（参看示例ParentChildTest.java）

```
class ParentClass {  
    public int myValue = 100;  
  
    public void printValue() {  
        System.out.println("Parent.printValue(),myValue=" + myValue);  
    }  
}  
  
class ChildClass extends ParentClass {  
    public int myValue = 200;  
  
    public void printValue() {  
        System.out.println("Child.printValue(),myValue=" + myValue);  
    }  
}
```



示例代码的特点是：
子类和父类定义了一模一样的
字段和方法

在实践中理解把握复杂的知识-2

运行以下测试代码，回答问题

```
public class ParentChildTest {  
    public static void main(String[] args) {  
        var parent = new ParentClass();  
        parent.printValue();  
        var child = new ChildClass();  
        child.printValue();  
  
        parent = child;  
        parent.printValue();  
  
        parent.myValue++;  
        parent.printValue();  
  
        ((ChildClass) parent).myValue++;  
        parent.printValue();  
    }  
}
```

问题：

1. 左边的程序运行结果是什么？
2. 你如何解释会得到这样的输出？
3. 计算机是不会出错的，之所以得到这样的运行结果也是有原因的，那么从这些运行结果中，你能总结出Java的哪些语法特性？

请务必先动脑总结，然后修改或编写一些代码进行测试，验证自己的想法，最后再看后面的PPT给出的结论。

总结（1）：子类父类拥有同名的方法时.....



当子类与父类拥有一样的方法，并且让一个父类变量引用一个子类对象时，到底调用哪个方法，由引用对象的“真实”类型所决定，这就是说：引用的对象是子类型的，它就调用子类型的方法，是父类型的，它就调用父类型的方法。



这个特性实际上就是面向对象“多态”特性的具体表现。



总结（2）：子类父类拥有同名同类型的字段时.....



如果子类与父类有相同的字段，则子类中的字段会代替或隐藏父类的字段，子类方法中访问的是子类中的字段（而不是父类中的字段）。如果子类方法确实想访问父类中被隐藏的同名字段，可以用`super`关键字来访问它。



然而，如果子类被当作父类使用，则通过子类访问的字段是父类的！



在实际开发中，要避免在子类中定义与父类同名的字段。不要自找麻烦！——但考试除外，考试中出这种题还是可以的。

super关键字-1

User类封装了某电商网站的用户信息，其中的discount字段代表他可享受的商品折扣，普通用户不能打折，VIP用户可以打折。

```
//某电商网站的用户打折信息
class User {
    private final String name;
    private final double discount;

    public User(String name) {
        //使用this调用私有构造函数，以便初始化折扣值为1.0
        this(name, discount: 1.0);
    }

    //私有构造函数，仅供自己调用
    private User(String name, double discount) {
        this.name = name;
        this.discount = discount;
    }

    public String getName() {...}
    public double getDiscount() {...}
}
```

super关键字-2

如果父类与子类拥有相同名字字段或方法，则子类可以通过super关键字引用或调用父类的同名方法。

注意一下子类VIPUser如何使用super关键字调用基类的构造方法和getDiscount方法。



```
//VIP用户也“是一种”普通用户
class VIPUser extends User {
    //VIP会员级别
    private String vipLevel = "白银";
    public VIPUser(String name, String vipLevel) {
        //使用super调用基类的构造方法
        super(name);
        this.vipLevel = vipLevel;
    }
    //VIP用户购买商品可以打折，因此此处重写基类的同名方法
    @Override
    public double getDiscount() {
        //使用super调用基类的同名方法
        var superDiscount :double = super.getDiscount();
        return switch (vipLevel) {
            case "白银" -> superDiscount * 0.8;
            case "黄金" -> superDiscount * 0.7;
            case "钻石" -> superDiscount * 0.6;
            default -> superDiscount;
        };
    }
    public String getVipLevel() {...}
}
```

示例运行截图

UseSupper.java

```
public class UseSupper {  
    public static void main(String[] args) {  
        var user = new User( name: "张三");  
        System.out.println(user.getName() + "的折扣为: " + user.getDiscount());  
  
        var vip = new VIPUser( name: "李四", vipLevel: "黄金");  
        System.out.println(vip.getName() + "是" + vip.getVipLevel()  
            + "会员, 享受的折扣为: " + vip.getDiscount());  
    }  
}
```

Run: UseSupper x



"C:\Program Files\Java\jdk-15\bin\java.exe"

张三的折扣为: 1.0

李四是黄金会员, 享受的折扣为: 0.7

Process finished with exit code 0

适用于学霸们的技术探索课题

我们已经知道多态代码是这么一回事：

当多个类实现同一接口（或派生自同一抽象类）时，针对这些类所创建的对象调用接口所定义的方法时，会分别调用相应的类的具体实现代码。



编译器在编译上述代码时，采用的方法是“**滞后绑定 (late bind)**”，因为它在编译时无法知道到底调用哪个方法。

请使用javap（或jClasslib之类反汇编工具）查看编译器为TestPolymorphism.java生成的字节码指令，然后通过互联网搜索资料，尝试从底层开始理解Java编译器是如何为多态代码生成字节码指令，在程序运行过程中，多态特性又是如何实现。

学而不思则罔



- 多态看上去蛮复杂的，那么，最初设计出它们的那些牛人们，到底是怎么想的？
- 多态对开发到底有什么好处呢？
- 为什么它被看成是面向对象编程技术中最重要的特性之一？

在实例中理解多态的含义与用途

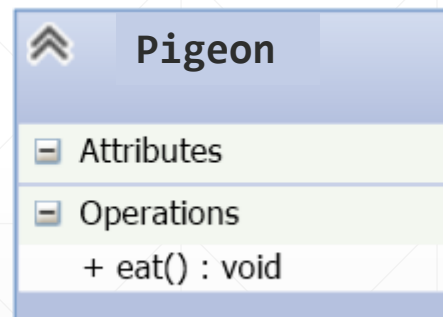
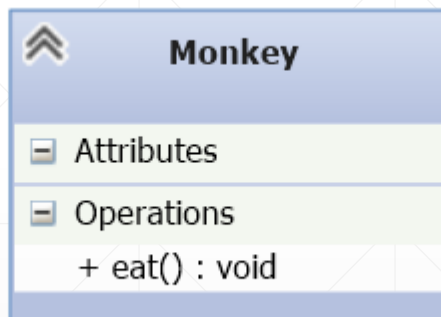
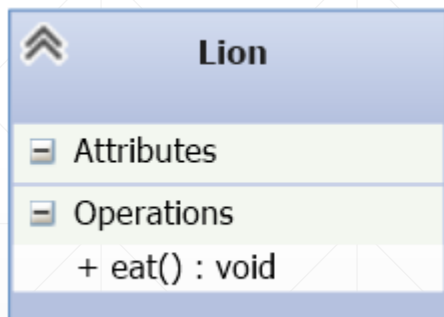
让我们看一个开发场景：

某动物园有一个饲养员小李，
每天需要给他所负责饲养的狮子、猴子和鸽子喂食。
请用一个程序来模拟他喂食的过程。



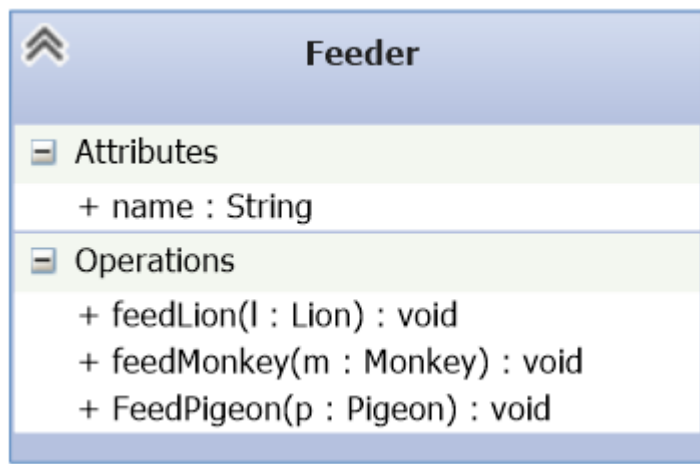
面向对象建模-1

- 三种动物对应三个类，每个类定义一个eat()方法，表示吃饲养员给它们的食物



面向对象建模-2

- 再设计一个Feeder类代表饲养员，其name字段保存饲养员名字，三个方法分别代表喂养三种不同的动物，其参数分别引用三种动物对象。



喂食过程

```
public class Zoo {  
    public static void main(String[] args) {  
        Feeder f = new Feeder( name: "小李");  
        // 饲养员小李喂养一只狮子  
        f.feedLion(new Lion());  
        // 饲养员小李喂养十只猴子  
        for (int i = 0; i < 10; i++) {  
            f.feedMonkey(new Monkey());  
        }  
        // 饲养员小李喂养5只鸽子  
        for (int i = 0; i < 5; i++) {  
            f.feedPigeon(new Pigeon());  
        }  
    }  
}
```

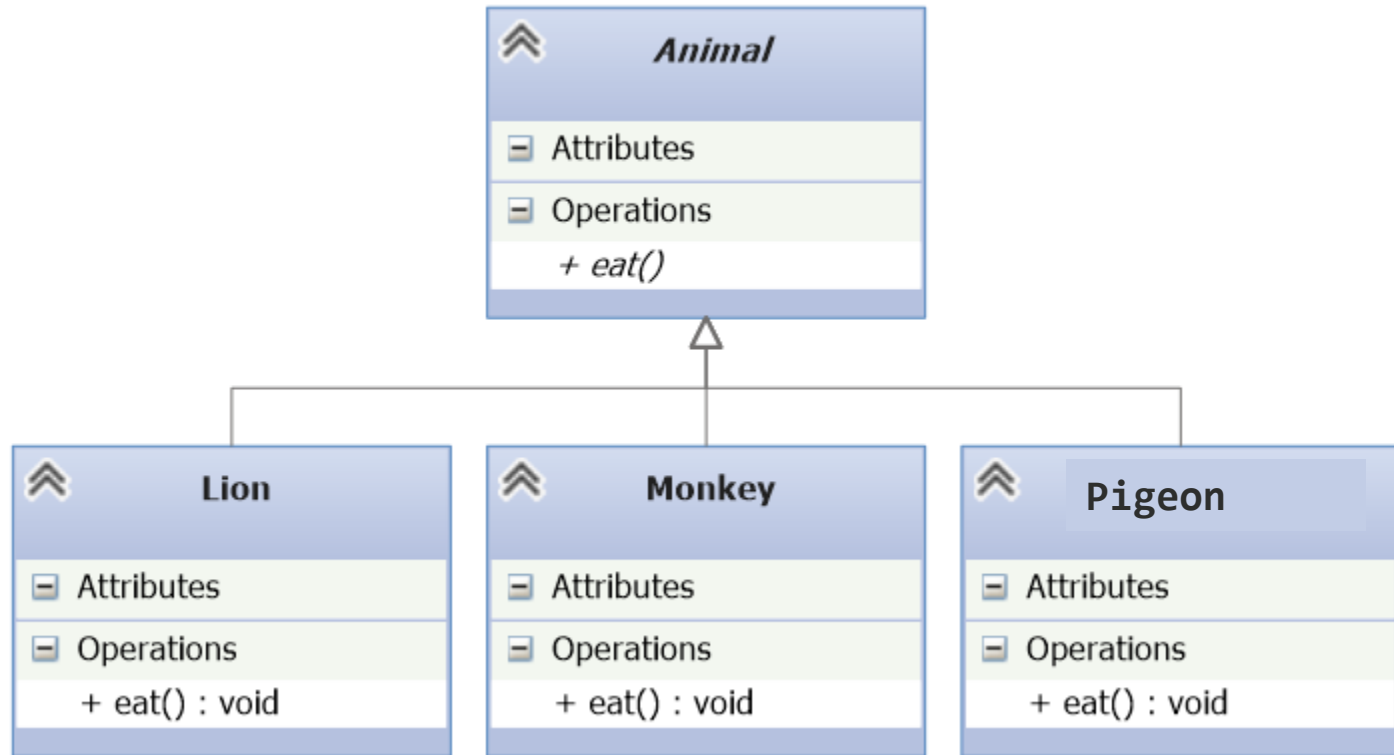
参看“动物园”示例版本一：Zoo1



思索：
这种编程方式有什么不合理的地方吗？

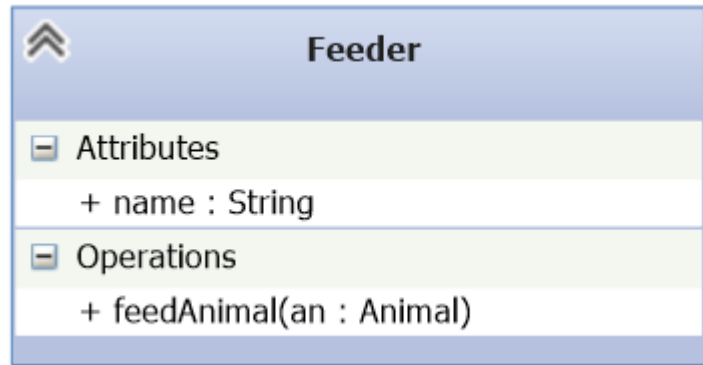
第一次程序重构(1): 引入继承

- 定义一个抽象基类Animal，其中定义一个抽象方法eat()，三个子类实现这个抽象方法。



第一次程序重构(2): 简化Feeder类

- Feeder类的三个喂养方法现在可以合并为一个feedAnimal()方法，注意它接收一个类型为Animal的对象类型参数，而不是三个具体的动物类型参数。依据多态特性，此方法将可以接收任何一个派生自Animal类的子类对象。



这种修改，将使Feeder类可用于代表任何一个饲养员，而不是仅仅适用于“小李”这样一个具体的饲养员。因为在现实生活中，每个饲养员饲养的动物，种类通常都是不一样的，不能为每一种动物，都对应一个特定的feedXXX()方法。

第一次重构结果

```
public class Zoo {  
  
    public static void main(String[] args) {  
        Feeder f = new Feeder( name: "小李");  
        //饲养员小李喂养一只狮子  
        f.feedAnimal(new Lion());  
        //饲养员小李喂养十只猴子  
        for (int i = 0; i < 10; i++) {  
            f.feedAnimal(new Monkey());  
        }  
        //饲养员小李喂养5只鸽子  
        for (int i = 0; i < 5; i++) {  
            f.feedAnimal(new Pigeon());  
        }  
    }  
}
```



还可以进一步优化……

“动物园”示例版本二: Zoo2

第二次重构

修改feedAnimal方法，让它接收一个Animal数组……

```
class Feeder {  
    //.....  
    //喂养一群动物  
    public void feedAnimals( Animal[] ans) {  
        foreach (Animal an : ans) {  
            an.eat();  
        }  
    }  
}
```

第二次重构结果

```
public class Zoo {  
  
    public static void main(String[] args) {  
        Feeder f = new Feeder( name: "小李");  
        //小李将要喂养16只动物  
        Animal[] ans = new Animal[16];  
  
        //需要饲养员小李喂养的一只狮子  
        ans[0] = new Lion();  
        //需要饲养员小李喂养的十只猴子  
        for (int i = 0; i < 10; i++) {  
            ans[1 + i] = new Monkey();  
        }  
        //需要饲养员小李喂养的5只鸽子  
        for (int i = 0; i < 5; i++) {  
            ans[11 + i] = new Pigeon();  
        }  
        //喂养动物  
        f.feedAnimals(ans);  
    }  
}
```

现在只需要一句代码，就可以模拟出喂养一批各种各样的动物的过程……

事情还不够完美

- 第二次重构之后，Feeder类的feedAnimals()方法接收的是一个Animal数组，这有一个限制，就是只能创建固定个数的数组，无法动态地增减动物个数。
- 想想以下场景：
 - (1) 动物园新进了一些动物
 - (2) 某动物生病不幸死亡
 - (3)
- 我们的代码能否应付以上的场景？

第三次重构

```
class Feeder {  
  
    public String name;  
  
    Feeder(String name) {  
        this.name = name;  
    }  
  
    //喂养一群动物  
    public void feedAnimals(List<Animal> ans) {  
        for (Animal an : ans) {  
            an.eat();  
        }  
    }  
}
```

修改feedAnimals方法，让其接收一个元素数目可变的对象集合

List是JDK所提供的的一个对象集合接口，实现了这一接口的集合，可以随时向它添加或删除元素。

第三次重构结果

```
public class Zoo {  
  
    public static void main(String[] args) {  
        Feeder f = new Feeder( name: "小李");  
        //饲养员小李需要喂养“一群”动物  
        List<Animal> ans = new ArrayList<>();  
  
        //需要饲养员小李喂养的一只狮子  
        ans.add(new Lion());  
        //需要饲养员小李喂养的十只猴子  
        for (int i = 0; i < 10; i++) {  
            ans.add(new Monkey());  
        }  
        //需要饲养员小李喂养的5只鸽子  
        for (int i = 0; i < 5; i++) {  
            ans.add(new Pigeon());  
        }  
        //喂养动物  
        f.feedAnimals(ans);  
    }  
}
```

这么重构以后，只要准备好一个类型为 `Animal` 的对象集合，就可以把它传给饲养员对象的 `feedAnimals()` 方法，不管有多少种类的动物，也不管某个特定饲养员要喂养多少只动物，饲养员类的代码都不需要修改。

本例中将集合的类型设置为基类，然后可以向其中添加各种子类对象，这种集合，称为“多态集合”。

“多态”小结-1



- 从这个示例中可以看到，通过在编程中应用多态，可以使我们的代码具有更强的适用性。当需求变化时，多态特性可以帮助我们将需要改动的地方减少到最低限度。
- 多态编程有两种主要形式：
 - (1) **继承多态**：示例程序使用的方法
 - (2) **接口多态**：使用接口代替抽象基类，这个任务留为作业。

“多态”小结-2



现在我们可以回答前面提出的问题了：

为什么要用多态？它有什么好处？

使用多态最大的好处是：

当你要修改程序并扩充系统时，你需要修改的地方**较少**，对其它部分代码的影响**较小**！

千万不要小看这两个“较”字！程序规模越大，其优势就越突出。

恭喜你完成了Java面向对象技术第一模块的学习！

再接再厉！